

Wrapping AWS Lambda ESM handler

Disclaimer:

This presentation and the code were heavily "vibe-coded" — proceed with good vibes and a cup of coffee ☕.



TL;DR

- Goal: wrap exported ESM `handler` functions with `wrapAwsLambda` automatically.
- Method: transform module source at import time (loader hooks or `LD_PRELOAD` detour).

Example (before → after)

Before:

```
export const handler = async (event) => ({ status: 200, body: 'ok' });
```

After:

```
export const handler = WrapAwsLambda(async (event) => ({ status: 200, body: 'ok' }));
```

Why: add observability hooks while preserving the original behavior of user code.

High-level approach

1. Parse source into an AST.
2. Traverse & transform the AST (find exports, replace node with wrapper call).
3. Generate code from the transformed AST.

This mirrors typical Babel-style transformations, but implemented in Rust with `oxc.rs` for performance and safety.

Implementation (Rust native addon)

We expose a simple function via `napi.rs` that takes source text and returns transformed source:

```
use napi_derive::napi;
mod transform;

#[napi]
pub fn transform_lambda(input: String) -> String {
    transform::transform_lambda_source(input)
}
```

Responsibilities:

- `transform_lambda_source` — decode, run AST transform, codegen, and return result.
- `LambdaTransform` — the AST walker/transformation logic (see next slides).

Core transform function

Key steps (simplified):

```
pub fn transform_lambda_source(source_text: String, handler: String, wrapper: String) -> String {  
    let allocator = Allocator::default(); // uses arena allocation  
    let parsed = Parser::new(&allocator, &source_text, SourceType::mjs()).parse();  
    let mut program = parsed.program;  
  
    let scoping = SemanticBuilder::new()  
        .build(&program)  
        .semantic  
        .into_scoping();  
  
    LambdaTransform::new(&allocator, handler, wrapper)  
        .transform(&allocator, &mut program, scoping);  
  
    Codegen::new().build(&program).code  
}
```

AST transform pattern (concept)

- Locate `ExportNamedDeclaration` nodes that export a `handler` binding.
- If the exported value is a function (declaration, arrow, or identifier pointing to a function), replace the exported value with a `CallExpression` to `WrapAwsLambda(...)` where the original function is passed in.

AST deep dive (example snippet)

```
impl<'a> Traverse<'a, ()> for LambdaTransform<'a> {  
    fn enter_program(&mut self, program: &mut Program<'a>, ctx: &mut TraverseCtx<'a, ()>) {  
        let mut new_stmts = ctx.ast.vec_with_capacity(program.body.len() * 2);  
        for stmt in program.body.drain(..) {  
            match &stmt {  
                Statement::ExportNamedDeclaration(export) => {  
                    // detect `export const handler = ...` and rewrite here  
                }  
                _ => (),  
            }  
            new_stmts.push(stmt);  
        }  
        program.body = new_stmts;  
    }  
}
```

This pattern drains the program body and rebuilds it with transformed statements — a simple and robust way to insert, remove, or replace nodes.

Usage (loader example)

```
import { registerHooks } from 'node:module';
import { transformLambda } from '../index.js';

registerHooks({
  load(url, ctx, next) {
    const res = next(url, ctx);
    if (url.endsWith('/handler.mjs')) {
      return { format: 'module', shortCircuit: true,
        source: transformLambda(res.source.toString(), 'handler', 'WrapAwsLambda') };
    }
    return res;
  }
});
```

To run the hook above use:

```
node --import ./hooks/sync-hooks-oxc.mjs runtime.mjs
```

Testing & Debugging

```
#[test]
fn test_var_transform() {
    let source_text = r#"
        export const handler = async function(event) {
            return "Hi from AWS Lambda";
        }, other = 123;
    "#.to_string();
    let expected_text = r#"export const handler = wrapper(async function(event) {
        return "Hi from AWS Lambda";
    }), other = 123;
    "#.to_string();
    let transformed = transform_lambda_source(source_text, "handler".to_string(), "wrapper".to_string());
    assert!(transformed.contains("wrapper"));
    assert!(transformed == expected_text);
}
```

Use unit tests (like the `test_var_transform` above) to validate transform input → expected output.

Frida detour — `open_detour` snippet

Intercept `open()` calls and transform `handler.mjs` content before Node reads it.

```
unsafe extern "C" fn open_detour(name: *const c_char, flags: c_int) -> c_int {
    let path = unsafe { std::ffi::CStr::from_ptr(name) }.to_str().unwrap();
    if path.ends_with("handler.mjs") {
        // call original open to get fd
        let fd = unsafe { ORIGINAL_OPEN.lock().unwrap().get().as_ref().unwrap().unwrap()(name, flags) };
        // read file, transform, and stash transformed contents for later reads
        let content = read_to_string(path).unwrap();
        STATE.lock().unwrap().transformed = transform::transform_lambda_source(
            content, "handler".to_string(), "WrapAwsLambda".to_string(),
        );
        STATE.lock().unwrap().handler_fd = fd;
        return fd;
    }
    // fallback to original
    unsafe { ORIGINAL_OPEN.lock().unwrap().get().as_ref().unwrap().unwrap()(name, flags) }
}
```

Frida detour installation

Minimal example (based on `detours.rs`):

```
static CELL: OnceLock<Gum> = OnceLock::new();
let gum = CELL.get_or_init(Gum::obtain);
let mut interceptor = Interceptor::obtain(gum);
unsafe {
    *ORIGINAL_OPEN.lock().unwrap().get_mut() = Some(std::mem::transmute(
        interceptor.replace(
            Module::load(gum, "libc.so.6").find_export_by_name("open").unwrap(),
            NativePointer(open_detour as *mut c_void),
            NativePointer(std::ptr::null_mut()),
        ).unwrap().0
    ));
}
```

Frida — LD_PRELOAD usage

Use `LD_PRELOAD` to load the native detours library before Node starts:

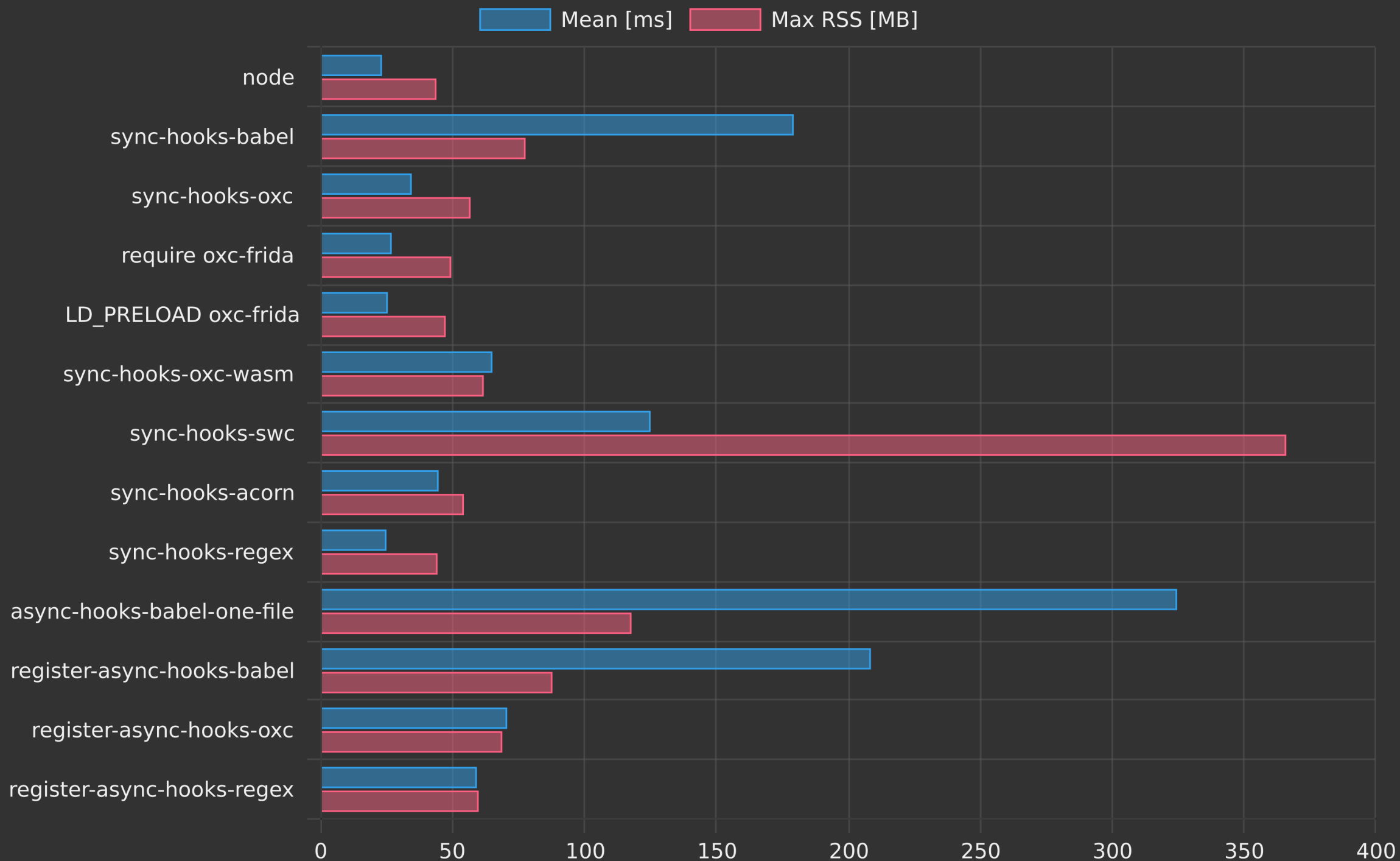
```
LD_PRELOAD=./wrap-esm-lambda.linux-x64-gnu.node node runtime.mjs
```

This injects the native library that installs the Frida hooks (`open`, `read`, etc.) so `handler.mjs` can be transformed at file-open time.

Benchmark

Benchmark table via [hyperfine](#) and `usr/bin/time -v` for Max RSS:

Hook	Mean [ms]	Min [ms]	Max [ms]	Relative	Max RSS [MB]
regex	24.3 ± 0.9	22.8	28.6	1.03 ± 0.06	44.24
LD_PRELOAD	25.8 ± 0.7	24.4	28.2	1.10 ± 0.06	49.26
oxc	35.6 ± 1.0	33.9	38.4	1.51 ± 0.09	54.92
acorn	45.0 ± 1.4	43.2	50.5	1.91 ± 0.11	56.30
swc plugin	127.4 ± 4.3	120.8	135.1	5.42 ± 0.33	371.61
babel	180.0 ± 4.0	172.1	188.2	7.66 ± 0.42	82.51
async babel	211.4 ± 4.2	205.9	220.3	9.00 ± 0.49	90.44



References

- oxc: <https://oxc.rs/>
- napi.rs: <https://napi.rs/>
- oxc playground: <https://playground.oxc.rs/>
- Frida: <https://frida.re/>
- frida-rust (Rust bindings): <https://github.com/frida/frida-rust>